

QuantAnomalies

What It Is and What Is Built

A statistical dashboard that hunts for market anomalies, fuses them into one verdict, and has a local AI explain the result — without letting the AI touch the numbers

Last updated 26 July 2026. Every figure below was recomputed from live code on that date.

1 The Idea in One Paragraph

Textbook finance says prices reflect everything, risk is constant, and returns are unpredictable. Decades of evidence say otherwise, and the mismatches are recurring, studied patterns called **anomalies**.

An anomaly is just an opportunity seen from the other side: the same crack academics flag as “the model is wrong here” is where a careful observer might find an edge. This project makes three such anomalies measurable, **fuses them into a single directional read**, and explains it.

Textbook says...	Reality (the opportunity)	Analogy
Prices react instantly	Macro news ripples over weeks	A stone in a pond — the ripple takes time to reach the edge
Risk is constant	Volatility clusters ; tails are fat	Storms come in clusters, and big ones hit more often than forecast
Returns are unpredictable	Some pairs are cointegrated	Two dogs on one leash — they wander, then snap back

2 Architecture



Stack: Python • FastAPI • pandas/numpy • statsmodels • arch • scipy • React 19 • Vite • Recharts •

framer-motion • llama.cpp. No frontend dependency was added for any of the recent work.

3 The Three Pillars (the signal sources)

Pillar 1 — Macro Factor & Lag Regression

Question: Which macro forces move an asset, and with what delay?

Method: Monthly OLS of the asset's return on eight factors (VIX, oil, gold, dollar index, 10-yr yield, Fed Funds, inflation, unemployment) at lags 0–3, plus Granger causality and a lagged-correlation heatmap. Factors are **z-score standardised** before fitting, so coefficients are comparable "standardised betas" — the effect per 1-SD move. VIX enters as a log-change for stationarity.

Current: $\hat{GSPC} R^2 = 0.687$ (adj. 0.585); Granger runs 32 tests, **6 significant**.

Pillar 2 — GARCH & Volatility Clustering

Question: How does risk change over time, and are crashes under-estimated?

Method: GARCH(1,1) with Student-t errors on daily returns; Ljung-Box on squared returns for clustering; Jarque-Bera and a QQ plot for normality.

Current: $\hat{GSPC}$ persistence = **0.9941** over 2,905 observations; excess kurtosis **15.8**, skew **-0.65**, Jarque-Bera $p \approx 0$ — decisively non-normal.

Pillar 3 — Forex Pair Trading

Question: Which currency pairs are tethered, and when is the gap tradeable?

Method: Engle-Granger cointegration across all pair combinations; for the best pair, a hedge-ratio spread on **log prices**, standardised to a z-score, with entry beyond ± 2 and exit near 0, plus a mean-reversion half-life.

Current: USDCHF/USDJPY, $p = 0.0075$, half-life **69 days**, hedge ratio -0.339, 91 historical signals.

Also present — Black-Scholes

European call/put pricing, the five Greeks, and a Newton/bisection implied-volatility solver, served at `/api/options/black-scholes`. It computes a `vol_verdict` (options rich or cheap versus the GARCH forecast) but nothing consumes it yet.

4 The Decision Engine

This is the layer that turns three separate reports into one answer. The pillars are untouched and still individually callable; the engine sits above them. **Seven detectors** feed it: volatility regime, tail event, macro dislocation, pairs opportunity, trend, breakout, and relative performance.

Normalisation

The detectors speak seven different direction vocabularies ("uptrend", "above_model", "long spread", "compressed"...). A polarity map collapses them onto one bull/bear/neutral axis so they can be compared at all. Volatility and tail readings are routed to a **separate risk axis**, so a volatility spike can never fake a directional call. Each signal also carries a **reliability** derived from its own statistics — cointegration p-value, macro R^2 , GARCH sample size — never invented.

Fusion

Weight $w = \text{severity} \times \text{reliability}$, then:

```

tilt          = (bull - bear)/(bull + bear)    direction, -1.. + 1
agreement     = 1 - min(bull, bear)/ max(bull, bear)    how one-sided
mass         = 1 - e-total/2    saturating evidence weight
conviction    = mass × (0.4 + 0.6 × agreement)

```

Strength beats count, nothing pins to 1.0 on noise, and a signal pointing the other way actively *lowers* conviction. This replaced an earlier $0.2 + 0.15n + 0.3$ top line that was dominated by signal count and clamped to 1.0 the moment six of anything showed up.

Tiered scan

Price detectors run broad and cheap; one cointegration sweep serves the whole FX basket; GARCH is cached on (ticker, last_data_date) so it refits once per trading day regardless of request volume. A detector that fails is recorded in diagnostics, never raised — one dead factor costs one signal, not the verdict. Measured warm: feed **11 ms**, per-asset **4.5 ms**, off-universe cold **0.9 s**. Endpoints: /api/engine/{feed,asset,narrate,status}, alongside the unchanged per-pillar routes used for drill-down.

5 The AI Layer — “stats detect, the LLM explains”

The model receives the computed detections as compact JSON and is instructed to use only those numbers. It never feeds back into the fusion. Three things make that verifiable rather than merely asserted:

- **Stance-first streaming.** The prompt demands a *STANCE:* line before the prose, so the model’s labelled opinion lands after ≈ 10 tokens and the reasoning streams beneath it instead of blocking for ≈ 20 s.
- **The model may disagree.** Its stance renders *beside* the computed stance, so a divergence is visible rather than hidden.
- **A number guardrail.** `unverified_numbers()` checks every figure in the narrative against the evidence supplied; anything unmatched surfaces in the UI as a grounding badge.

Runtime is a local llama.cpp server (Qwen3-4B) on an RTX 4050 via Vulkan, with an in-process CPU fallback and a rules-only path if no model is up. Nothing leaves the machine.

6 Data Pipeline

Yahoo Finance for prices, FRED for macro, cached as CSV so the app runs offline once fetched. What we hardened, and why:

- FRED was returning **HTTP 403** — fixed with a proper request header, plus a **DBnomics** mirror as fallback. The flaky 10-yr series was swapped for Yahoo’s ^{^TNX}.
- `build_macro_dataset` is **fault-tolerant**: a failed factor is skipped, not fatal.
- **yfinance is not thread-safe.** Concurrent `yf.download()` calls silently return one ticker’s data under another’s name — a threaded scan reproduced it instantly (META/TSLA, JPM/GLD, ^{^IXIC}/^{^DJI} came back byte-identical). This was the root cause of the “corrupted cache” we had previously only patched. Downloads are now serialized behind a lock; cached reads stay parallel.
- **Freshness:** the end date had been hardcoded to 2025-12-31 and caches never expired, so the dashboard was reasoning over seven-month-old data. Now a rolling end date plus an mtime staleness check, with a failed refresh falling back to the stale copy rather than to nothing.

- **run_granger_causality was silently dead** — it passed a verbose kwarg removed in statsmodels 0.14, and a bare except swallowed the TypeError, so it always returned zero results. Fixed.

7 The Dashboard

Design rule: colour means direction, and nothing else gets to use it. Bull/bear are the only hues. Severity is bar length, source is a label, risk is a segmented meter.

- **Two-tier readout.** Every metric gives a plain-English word in the primary slot, the exact figure beside it, and the mechanism in a hover tip. One element serves a beginner and a trader; nothing needed to read the screen is behind a click.
- **Tilt gauge** — one instrument replacing four readouts: marker = balance of evidence, band width = inverse conviction, so it reads like a confidence interval.
- **Dissent is visible.** Signals opposing the verdict render under “Pushing the other way” instead of being averaged away.
- **Drill-down.** A signal’s source badge jumps to the pillar tab that produced it.
- **Dual theme.** Light by default (it survives a projector), dark follows the OS, three-state toggle. All 30 token pairings verified at $\geq 4.5:1$ contrast in both.
- **Type & layout.** Archivo (variable) + JetBrains Mono; five sizes replacing 32 ad-hoc values; bento grids that tile exactly at every breakpoint; signals as a hairline list rather than a card grid.
- **Shared ticker** across all five tabs via context — panels used to hold a private `useState("^GSPC")`, so switching tabs reset the asset.
- **Skeletons** matched to the layout on first load, dim-in-place on refetch; a session cache (stale-while-revalidate) removed the jank on tab switches.
- **Status strip** reads `/api/engine/status`: live / warming / offline. The dot used to be hardcoded green, including when the backend was down.
- **Time-range filters** (1M–All) on every time series, y-axis autoscaling to the visible window.

8 Testing & Verification

- `backend/test_engine.py` — 25 cases over the polarity map, fusion invariants, ranking stability, reliability derivation, stance parsing, and the number guardrail. These are the first tests in the repo.
- An unknown ticker used to return **500** quoting an internal cache key. `data_loader` now raises `NoDataError` and the API answers **404** with a readable message — one handler covers every endpoint, since all data access routes through `_load_or_download`.
- `npm run lint` is clean. Design-hook detector: 8 findings → 0. Contrast verified in both themes.

9 What’s Next

The largest remaining gap is **statistical rigor**: the detectors are built but several inferential claims are unvalidated — multiple-testing correction on the 990 cointegration tests, HAC/Newey-West errors on the autocorrelated monthly regression, differencing the level-valued macro series, out-of-sample GARCH VaR with a Kupiec test, and removing look-ahead bias from the pair-signal gener-

ation. After that: wiring the Black–Scholes vol-mispricing detector into the engine, news-on-chart-click, and a backtest that turns “opportunities” into measured evidence.

10 How to Run

- **Backend:** `python -m uvicorn main:app --app-dir backend --port 8000` (use the Anaconda interpreter on this machine).
- **Local AI (optional but recommended):** `llama-server.exe -m <Qwen3-4B gguf> -ngl 99 -c 4096 --port 8080`
- **Frontend:** `npm run dev` in `frontend/` → open `http://localhost:5173`.

Without the AI server the app runs fine — narration falls back to in-process CPU (≈ 5.7 s) and then to a rules-only summary.